Chalenge 03 - Relatório de Testes – API ServeRest

Relatório de Execução de Testes – API ServeRest Squad 3 - Bug Hunters

Relatório de Execução de Testes – API ServeRest Squad 3 - Bug Hunters
1. Sumário Executivo
2. Evidências de Execução
2.1 Módulo: Autenticação & Usuários
TC-01 - [POST] - LOGIN - Validar autenticação com credenciais válidas e extração de Bearer Token
TC-02 - [POST] LOGIN - Validar bloqueio de acesso ao informar senha incorreta.
TC-03 - [POST] - LOGIN Validar integridade do contrato (Schema) da resposta de Login.
TC-04: [POST] - Usuários - Positivo - Validar criação de novo usuário com dados válidos e persistência de ID.
TC-06: [POST] - Usuário - Contrato (Opcional - Validação de Campos)
TC-07: [GET] - Usuários - Positivo - (Listagem)
TC-08: [GET] - Usuário - Positivo - (Buscar por ID)
TC-09: [GET] - Usuário - Negativo - (ID inexistente)
TC-10: [PUT] - Usuário - Positivo - (Atualização)
TC-11: [PUT] - Usuário - Negativo - (Email já cadastrado)
TC-12: [DELETE] - Usuário - Positivo - (Exclusão)
TC-13: [DELETE] - Usuário - Negativo - (ID inexistente/Inválido)
2.2 Módulo: Produtos & Carrinhos
TC-14: [POST] - Produto - Positivo - (Cadastro válido)
TC-15: [POST] - Produto - Negativo - (Produto com preço inválido)
TC-16: [GET] - Produtos - Positivo - (Listagem)
TC-17: [GET] - Produto - Positivo - (Buscar por ID)
TC-18: [GET] - Produto - Negativo - (ID inexistente)
TC-19: [PUT] - Produto - Positivo - (Atualização)
TC-20: [PUT] - Produto - Negativo - (Usuário sem permissão)
TC-21: [DELETE] - Produto - Positivo - (Exclusão)
TC-22: [DELETE] - Produto - Negativo - (Produto vinculado a carrinho)
TC-23: [POST] - Carrinho - Positivo - Criar carrinho
TC-24: [POST] - Carrinho - Negativo - Carrinho já existente
TC-26: [POST] - Carrinho | Negativo - Estoque insuficiente
TC-27: [GET] - Carrinhos | Positivo - Listar carrinhos
TC-28: [GET] - Carrinho | Positivo - Buscar por ID
TC-29: [DELETE] - Carrinho | Positivo - Concluir compra
TC-30: [DELETE] - Carrinho | Positivo - Cancelar compra
TC-31: [DELETE] - Carrinho | Negativo - Nenhum carrinho ativo
3. Gestão de Bugs
BUG-01: Cadastro de produto aceita preço como String (POST /produtos)
BUG-05: Falha de Validação - (Dados inválidos/Campos vazios)
BUG-12 Atualização de usuário permite nome e password vazios via método PUT
BUG-04: Atualização de Produto aceita String em campos Numéricos (PUT /produtos)
Passos para reproduzir:
Resultado Esperado:
Resultado Obtido (BUG):
BUG-25: Criação de carrinho aceita quantidade tipo String (POST /carrinhos)
Passos para reproduzir:
Resultado Esperado:
Resultado Obtido (BUG):
4. Checks Automatizados
5. Recomendações e Considerações Finais

1. Sumário Executivo

Objetivo do Documento

Registrar a execução dos testes planejados para a API ServeRest, documentando evidências, resultados observados e defeitos identificados, com rastreabilidade para os tickets registrados no Jira.

Item	Quantidade / Status
Total de Cenários Planejados	32 (TC-01 a TC-32)
Cenários Executados	100%
Passou PASSED	29
Falhou FAILED/BUG	05
Status Final do Ciclo	🔴 Reprovado (Bugs Críticos Identificados)
Autor	Arlen Freitas

---

2. Evidências de Execução

A execução dos testes foi realizada manualmente utilizando o Postman, com foco nos cenários críticos definidos no plano de testes. Foram priorizados cenários positivos, negativos e de validação de contrato, garantindo a cobertura mínima necessária para avaliação funcional e de regras de negócio da API.

2.1 Módulo: Autenticação & Usuários

TC-01 - [POST] - LOGIN - Validar autenticação com credenciais válidas e extração de Bearer Token

Severidade: Crítica.

Descrição: Verificar se a API permite o acesso de um usuário cadastrado e retorna o token de autorização necessário para as demais operações do sistema.

URL/Endpoint: `{{baseUrl}}/login`

Test Steps (Passos):

1. Configurar o método como **POST**.
2. No Body, inserir e-mail e senha de um usuário existente.
3. Executar a requisição e validar o status code e o corpo da resposta.

```
{ "email": "fulano@qa.com", "password": "teste" }
```

Resultado Esperado: HTTP 200 OK, mensagem "Login realizado com sucesso" e presença da chave `authorization` no JSON.

TC-02 - [POST] LOGIN - Validar bloqueio de acesso ao informar senha incorreta.

Severidade: Alta.

Descrição: Garantir que o sistema impeça a autenticação quando a senha não coincide com o e-mail informado, retornando erro de autorização.

URL/Endpoint: `{{baseUrl}}/login`.

Test Steps (Passos):

1. Configurar o método como **POST**.
2. Inserir um e-mail válido mas uma senha propositalmente incorreta.
3. Executar e validar a mensagem de erro.

```
{
    "email": "fulano@qa.com",
    "password": "senha_incorreta"
}
```

Resultado Esperado: HTTP 401 Unauthorized e mensagem "Email e/ou senha inválidos".

TC-03 - [POST] - LOGIN Validar integridade do contrato (Schema) da resposta de Login.

Severidade: Média.

Descrição: Assegurar que a estrutura do JSON retornado, os tipos de dados e os campos obrigatórios estão em conformidade com a documentação Swagger.

URL/Endpoint: `{{baseUrl}}/login`.

Test Steps (Passos):

1. Realizar uma chamada de login bem-sucedida.
2. Validar se a chave `message` é uma String.
3. Validar se o campo `authorization` começa com o prefixo "Bearer ".

```
{ "email": "fulano@qa.com", "password": "teste" }
```

Resultado Esperado: Sucesso nas asserções de contrato: campos presentes, tipos de dados corretos e formato do token válido.

TC-04: [POST] - Usuários - Positivo - Validar criação de novo usuário com dados válidos e persistência de ID.

Severidade: Alta.

Descrição: Verificar se a API permite o cadastro de um novo usuário (administrador ou comum) e se retorna o identificador único (`_id`) para uso em operações futuras.

URL/Endpoint: `{{baseUrl}}/usuarios`

Test Steps (Passos):

1. Configurar o método como **POST**.
2. No Body, inserir um objeto JSON com nome, e-mail único, senha e status de administrador.
3. Executar a requisição e validar se o ID foi gerado e armazenado na variável `userId`.

```
{
  "nome": "fulano qa",
  "email": "{{dynamicEmail}}",
  "password": "teste",
  "administrador": "true"
}
```

Resultado Esperado: HTTP 201 Created, mensagem "Cadastro realizado com sucesso" e recebimento de um `_id` válido.

TC-06: [POST] - Usuário - Contrato (Opcional - Validação de Campos)

Severidade: Média.

Descrição: Assegurar que a resposta de erro ou sucesso siga a estrutura de campos (`message` , `_id`) e tipagens definidas no Swagger.

URL/Endpoint: `{{baseUrl}}/usuarios`

Test Steps (Passos):

1. Realizar uma chamada de cadastro válida.
2. Validar se o campo `_id` retornado é do tipo String e não nulo.
3. Validar se a mensagem de sucesso está presente.

Resultado Esperado: Conformidade total com o Schema da API, sem campos ausentes ou com tipagem divergente.

TC-07: [GET] - Usuários - Positivo - (Listagem)

Severidade: Média.

Descrição: Verificar se o endpoint retorna a lista completa de usuários, incluindo a quantidade total e o array de objetos com seus respectivos dados.

URL/Endpoint: `{{baseUrl}}/usuarios`

Test Steps (Passos):

1. Configurar o método como **GET**.
2. Garantir que não existam parâmetros de busca (Params) ativos para uma listagem completa.
3. Executar a requisição e validar o Status Code e a estrutura do corpo da resposta.

```
N/A (Listagem Geral).
```

Resultado Esperado: HTTP 200 OK, campo `quantidade` maior ou igual a 0 e campo `usuarios` contendo um array de objetos.

TC-08: [GET] - Usuário - Positivo - (Buscar por ID)

Severidade: Alta.

Descrição: Confirmar que, ao informar um `_id` válido e existente, a API retorna os detalhes corretos daquele usuário específico.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id`.

Test Steps (Passos):

1. Configurar o método como **GET**.
2. Passar o valor de um ID válido (previamente capturado no TC-04) como **Path Variable**.
3. Executar e comparar se os dados retornados coincidem com o usuário cadastrado.

```
_id (ex: 0uxuPY0cbmQhpEz1).
```

Resultado Esperado: HTTP 200 OK e exibição dos campos `nome`, `email`, `password`, `administrador` e `_id` do usuário solicitado.

TC-09: [GET] - Usuário - Negativo - (ID inexistente)

Severidade: Média

Descrição: Garantir que o sistema trate adequadamente buscas por IDs que não constam na base de dados, retornando a mensagem de erro padrão.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id`.

Test Steps (Passos):

1. Configurar o método como **GET**.
2. Inserir um ID aleatório ou inexistente no campo da variável de path.
3. Executar e validar se o Status Code e a mensagem de erro seguem o padrão.

```
_id inexistente (ex: 0uxuPY0cbmQhpEz2
```

Resultado Esperado: HTTP 400 Bad Request (conforme comportamento da ServeRest) e mensagem "Usuário não encontrado".

TC-10: [PUT] - Usuário - Positivo - (Atualização)

Severidade: Alta.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id`.

Test Steps (Passos):

1. Configurar o método como **PUT**.
2. Informar o `_id` do usuário no path da URL.
3. No Body, inserir o JSON com os novos dados (ex: alteração do e-mail ou nome).
4. Executar e validar o status code e a mensagem de confirmação.

```
{
  "nome": "Obi-Wan",
  "email": "obi-wan@qa.com",
  "password": "Obi123",
  "administrador": "true"
}

```

Resultado Esperado: HTTP 200 OK e mensagem "Registro alterado com sucesso".

TC-11: [PUT] - Usuário - Negativo - (Email já cadastrado)

Severidade: Média.

Descrição: Garantir que o sistema bloqueie a alteração de e-mail de um usuário caso o novo endereço já esteja em uso por outra conta no banco de dados.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id`

Test Steps (Passos):

1. Configurar o método como **PUT**.
2. Informar o `_id` de um usuário.
3. No Body, inserir um e-mail que comprovadamente já existe em outro cadastro.
4. Executar e validar a resposta de erro.

```
{
  "nome": "Obi-Wan Editado",
  "email": "{{dynamicEmail}}",
  "password": "nova_senha123",
  "administrador": "true"
}
```

Resultado Esperado: HTTP 400 Bad Request e mensagem "Este email já está sendo usado".

TC-12: [DELETE] - Usuário - Positivo - (Exclusão)

Severidade: Média.

Descrição: Confirmar que o sistema remove o registro do usuário quando solicitado pelo método DELETE e retorna a confirmação de sucesso.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id.`

Test Steps (Passos):

1. Configurar o método como **DELETE**.
2. Informar o `_id` do usuário que deve ser removido.
3. Executar a requisição.

```
_id (ex: 0uxuPY0cbmQhpEz1).
```

Resultado Esperado: HTTP 200 OK e mensagem "Registro excluído com sucesso | Nenhum registro excluído".

TC-13: [DELETE] - Usuário - Negativo - (ID inexistente/Inválido)

Severidade: Baixa/Média.

Descrição: Garantir que o sistema trate corretamente a tentativa de remoção de um usuário que não consta na base de dados, retornando a mensagem apropriada de que nenhum registro foi afetado.

URL/Endpoint: `{{baseUrl}}/usuarios/:_id`

Test Steps (Passos):

1. Configurar o método como **DELETE**.
2. Informar um `_id` que não existe no sistema (ex: `ID_QUALQUER_123`).

3. Executar a requisição.
4. Validar se o status code é 400(comportamento padrão da ServeRest para este caso).

```
_id (ex: 0uxuPY0cbmQhpEz2)
```

Resultado Esperado: HTTP 400 e mensagem `"Não é permitido excluir usuário com carrinho cadastrado"` "Registro excluído com sucesso | Nenhum registro excluído".

2.2 Módulo: Produtos & Carrinhos

TC-14: [POST] - Produto - Positivo - (Cadastro válido)

- **Severidade:** Alta.
- **Descrição:** Garantir que um usuário autenticado com permissão de administrador consiga inserir novos itens no catálogo da loja.
- **URL/Endpoint:** `{{baseUrl}}/produtos`.
- **Test Steps (Passos):**
  a. Configurar o método como **POST**.
  b. Adicionar o `authToken` de um administrador no header `Authorization`.
  c. No Body, inserir o JSON com nome (único), preço, descrição e quantidade.
  d. Executar e validar a geração do `_id`.
-
```
{
  "nome": "Produto String",
  "preco": "100",
  "descricao": "Teste",
  "quantidade": 10
}
```

  Resultado Esperado: HTTP 201 Created, mensagem "Cadastro realizado com sucesso" e armazenamento do `_id` na variável `productId`.

TC-15: [POST] - Produto - Negativo - (Produto com preço inválido)

Severidade: Alta

Descrição:
Ao realizar o cadastro de um produto, o campo **preço** aceita valores entre aspas (String), quando o contrato define que o campo deve ser estritamente numérico. A API realiza uma coerção silenciosa ou aceita o dado incorreto, retornando **201 Created** em vez de validar o tipo de dado.

Test Steps (Passos):

- Acessar o endpoint POST /produtos.

- Utilizar um token de administrador válido no Header Authorization.
- Enviar o seguinte payload:

```
{
 "nome": "Produto Teste",
 "preco": "100",
 "descricao": "Teste",
 "quantidade": 10
}
```

Resultado Esperado: HTTP 400 Bad Request com mensagem indicando que o campo preco deve ser um número.

TC-16: [GET] - Produtos - Positivo - (Listagem)

- **Severidade:** Média.
- **Descrição:** Verificar se o endpoint retorna todos os produtos cadastrados no catálogo, exibindo a quantidade total e o array detalhado de itens.
- **URL/Endpoint:** `{{baseUrl}}/produtos`.
- **Test Steps (Passos):**
  a. Configurar o método como **GET**.
  b. Remover qualquer filtro de busca nos Query Params para garantir a listagem completa.
  c. Executar a requisição e validar o status code e a estrutura do JSON.
-
```
N/A (Listagem Geral).
```

  Resultado Esperado: HTTP 200 OK, campo `quantidade` presente e array `produtos` contendo objetos com `nome`, `preco`, `descricao`, `quantidade` e `_id`.

TC-17: [GET] - Produto - Positivo - (Buscar por ID)

- **Severidade:** Alta.
- **Descrição:** Confirmar que a API retorna as informações precisas de um produto quando um identificador único válido é fornecido no path.
- **URL/Endpoint:** `{{baseUrl}}/produtos/:_id`.
- **Test Steps (Passos):**

1. Configurar o método como **GET**.
2. Informar um `_id` válido (capturado durante o cadastro no TC-14) no campo da variável de path.
3. Executar a requisição e validar os dados retornados.

-
```
_id de produto existente (ex: BeeJh5lz3k6kSIzA).
```

Resultado Esperado: HTTP 200 OK e exibição dos atributos corretos do produto correspondente ao ID.

TC-18: [GET] - Produto - Negativo - (ID inexistente)

- **Descrição:** Garantir que o sistema informe adequadamente quando um produto não é localizado na base de dados, retornando o código de erro e mensagem padrão.
- **Severidade:** Baixa.
- **URL/Endpoint:** `{{baseUrl}}/produtos/:_id`.
- **Test Steps (Passos):**

1. Configurar o método como **GET**.
2. Inserir um valor de ID inexistente ou aleatório na URL.
3. Executar e validar a resposta de erro da API.

```
_id inexistente.
```

Resultado Esperado: HTTP 400 Bad Request e mensagem "Produto não encontrado".

TC-19: [PUT] - Produto - Positivo - (Atualização)

Severidade: Alta.

Descrição: Verificar se a API permite a alteração de atributos (nome, preço, descrição, quantidade) de um produto através de seu `_id`.

URL/Endpoint: `{{baseUrl}}/produtos/:_id`.

Test Steps (Passos):

1. Configurar o método como **PUT**.
2. Informar um `_id` de produto válido e o Token de administrador.
3. No Body, inserir o JSON com os novos valores para os campos do produto.
4. Executar e validar a mensagem de sucesso.

```
{
  "nome": "{{productName}}",
  "preco": 470,
  "descricao": "Mouse",
  "quantidade": 381
}
```

Resultado Esperado: HTTP 200 OK e mensagem "Registro alterado com sucesso".

TC-20: [PUT] - Produto - Negativo - (Usuário sem permissão)

Severidade: Crítica.

Descrição: Garantir que usuários comuns não consigam realizar edições no catálogo de produtos, protegendo a integridade do inventário.

URL/Endpoint: `{{baseUrl}}/produtos/:_id`.

Test Steps (Passos):

1. Configurar o método como **PUT**.
2. Informar um `_id` válido, porém utilizar o Token de um usuário comum (`administrador: false`).
3. Tentar enviar o payload de atualização.

Resultado Esperado: HTTP 401 Unauthorized (ou 403 Forbidden conforme implementação) e mensagem "Rota exclusiva para administradores".

TC-21: [DELETE] - Produto - Positivo - (Exclusão)

Severidade: Alta.

Descrição: Confirmar que o sistema remove o registro do produto ao receber uma solicitação DELETE autorizada.

URL/Endpoint: `{{baseUrl}}/produtos/:_id`.

Test Steps (Passos):

1. Configurar o método como **DELETE**.
2. Informar o `_id` de um produto existente e o Token de administrador.
3. Executar a requisição.

Resultado Esperado: HTTP 200 OK e mensagem "Registro excluído com sucesso | Nenhum registro excluído".

TC-22: [DELETE] - Produto - Negativo - (Produto vinculado a carrinho)

Severidade: Crítica.

Descrição: Verificar se a API protege a integridade transacional, impedindo a remoção de itens que estão sendo comprados.

URL/Endpoint: `{{baseUrl}}/produtos/:_id`.

Test Steps (Passos):

1. Certificar-se de que o produto com o `_id` informado está inserido em pelo menos um carrinho ativo.
2. Tentar realizar o DELETE utilizando Token de administrador.

3. Validar se a API bloqueia a exclusão.

Resultado Esperado: HTTP 400 Bad Request e mensagem "Não é permitido excluir produto que faz parte de carrinho".

TC-23: [POST] - Carrinho - Positivo - Criar carrinho

Severidade: Alta.

Descrição: Verificar se a API permite vincular produtos ao usuário do token, calculando o valor total e reservando os itens no estoque.

URL/Endpoint: `{{baseUrl}}/carrinhos`.

Test Steps (Passos):

1. Configurar o método como **POST** e adicionar o `token` no Header.
2. No Body, informar o `idProduto` (variável `{{productId}}`) e a quantidade desejada.
3. Executar a requisição e validar o Status Code 201.

```
{
  "produtos": [
    {
      "idProduto": "{{productId}}",
      "quantidade": 1
    }
  ]
}
```

Resultado Esperado: HTTP 201 Created e mensagem "Cadastro realizado com sucesso".

TC-24: [POST] - Carrinho - Negativo - Carrinho já existente

Severidade: Média.

Descrição: Garantir que a API respeite a regra de negócio de permitir apenas 1 carrinho ativo por usuário.

URL/Endpoint: `{{baseUrl}}/carrinhos`.

Test Steps (Passos):

1. Tentar realizar um novo POST em `/carrinhos` utilizando o mesmo token do TC-23.
2. Validar se a API bloqueia a criação duplicada.

Resultado Esperado: HTTP 400 Bad Request e mensagem contendo "Não é permitido ter mais de 1 carrinho".

TC-26: [POST] - Carrinho | Negativo - Estoque insuficiente

Severidade: Crítica.

Descrição: Garantir a integridade do inventário, impedindo que o estoque de um produto fique negativo.

URL/Endpoint: `{{baseUrl}}/carrinhos`.

```
{
  "produtos": [
    {
      "idProduto": "{{productId}}",
      "quantidade": 9999
    }
  ]
}
```

Resultado Esperado: HTTP 400 Bad Request e mensagem "Produto não possui quantidade suficiente".

TC-27: [GET] - Carrinhos | Positivo - Listar carrinhos

Severidade: Média.

Descrição: Verificar se o endpoint retorna a relação de todos os carrinhos cadastrados, exibindo a quantidade total e os detalhes de cada transação (produtos, preços e usuários).

URL/Endpoint: `{{baseUrl}}/carrinhos`.

Test Steps (Passos):

1. Configurar o método como **GET**.
2. Certificar-se de que nenhum filtro (Query Params) esteja ativo para garantir a listagem completa.
3. Executar a requisição e validar o Status Code e a estrutura do array retornado.

```
N/A (Listagem Geral).
```

Resultado Esperado: HTTP 200 OK, campo `quantidade` presente e array `carrinhos` contendo a estrutura de produtos e valores totais.

TC-28: [GET] - Carrinho | Positivo - Buscar por ID

Severidade: Alta.

Descrição: Garantir que, ao informar um `_id` de carrinho válido, a API retorne os detalhes precisos dos produtos reservados, preço total e o ID do usuário proprietário.

URL/Endpoint: `{{baseUrl}}/carrinhos/:_id`.

Test Steps (Passos):

1. Configurar o método como **GET**.
2. Informar um `_id` de carrinho válido (armazenado na variável `cartId`) no path da URL.
3. Executar a requisição e validar se os dados correspondem ao carrinho criado anteriormente.

```
_id de carrinho existente (ex: qbMqntef4iTOwWfg)
```

Resultado Esperado: HTTP 200 OK e exibição dos campos `produtos`, `precoTotal`, `quantidadeTotal`, `idUsuario` e `_id`.

TC-29: [DELETE] - Carrinho | Positivo - Concluir compra

Severidade: Crítica

Descrição: Verificar se o endpoint de conclusão de compra remove o carrinho vinculado ao token do usuário, consolidando a transação na base de dados.

URL/Endpoint: `{{baseUrl}}/carrinhos/concluir-compra`.

- **Test Steps (Passos):**
  a. Configurar o método como **DELETE**.
  b. Adicionar o `authToken` de um usuário que possua um carrinho ativo no Header.
  c. Executar a requisição e validar a mensagem de exclusão de sucesso.
- Token de usuário com carrinho ativo.

Resultado Esperado: HTTP 200 OK e mensagem "Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário".

TC-30: [DELETE] - Carrinho | Positivo - Cancelar compra

Severidade: Alta

Descrição: Garantir que o cancelamento remova o carrinho e retorne as quantidades dos produtos reservados para o estoque geral da loja.

URL/Endpoint: `{{baseUrl}}/carrinhos/cancelar-compra`.

Test Steps (Passos):

1. Configurar o método como **DELETE**.
2. Utilizar o `authToken` de um usuário com carrinho ativo.
3. Executar a requisição e validar se os itens retornaram ao estoque (via GET Produtos).

Token de usuário com carrinho ativo.

Resultado Esperado: HTTP 200 OK e mensagem "Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário".

TC-31: [DELETE] - Carrinho | Negativo - Nenhum carrinho ativo

Severidade: Baixa.

Descrição: Verificar se o sistema trata adequadamente a tentativa de exclusão de carrinho para usuários que não possuem compras em aberto.

URL/Endpoint: `{{baseUrl}}/carrinhos/cancelar-compra`.

Test Steps (Passos):

1. Configurar o método como **DELETE**.
2. Utilizar o `authToken` de um usuário que **não** possua carrinho (ou que já tenha concluído/cancelado).
3. Executar e validar a mensagem de retorno.

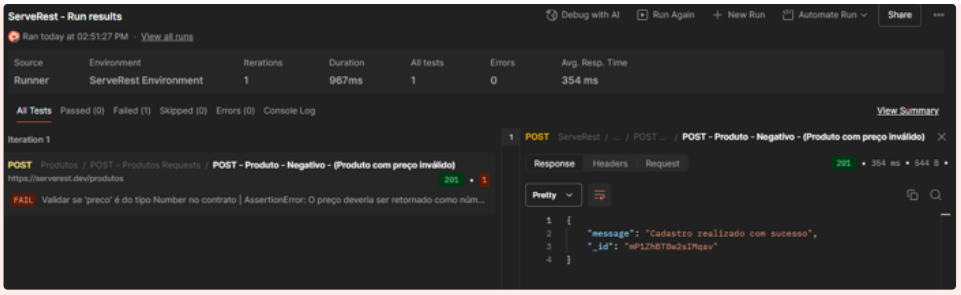
Token de usuário sem carrinho ativo.

Resultado Esperado: HTTP 200 OK e mensagem "Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário".

3. Gestão de Bugs

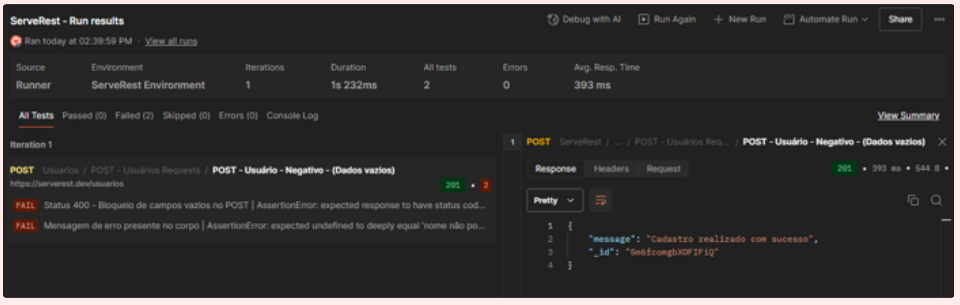
BUG-01: Cadastro de produto aceita preço como String (POST /produtos)

- **Severidade:** Alta.
- **Descrição:**
  Ao realizar o cadastro de um produto, o campo **preço** aceita valores entre aspas (String), quando o contrato define que o campo deve ser estritamente numérico. A API realiza uma coerção silenciosa ou aceita o dado incorreto, retornando **201 Created** em vez de validar o tipo de dado.
- **Passos para reproduzir:**
  1. Login Admin → 2. POST /produtos enviando `"preco": "100"` (String).
- **Resultado Esperado:**
- **Status Code:** 400 Bad Request (Erro de tipagem).
- **Resultado Obtido(Bug):**
- **Status Code:** 201 Created (Aceitou String em campo numérico).



BUG-05: Falha de Validação - (Dados inválidos/Campos vazios)

- **Severidade:** Crítica.
- **Descrição:**
  Verificar se a API rejeita a criação de um novo registo quando o `nome` ou a `password` são enviados como strings vazias, garantindo a obrigatoriedade dos dados conforme a documentação.
- **Passos para reproduzir:**
  1. POST /usuarios com `nome: ""` e `password: ""`.
- **Resultado Esperado:**
- **Status Code**: 400 Bad Request.
- **Resultado Obtido(Bug):**
- **Status Code**: 201 Created (Criação de usuário sem credenciais).

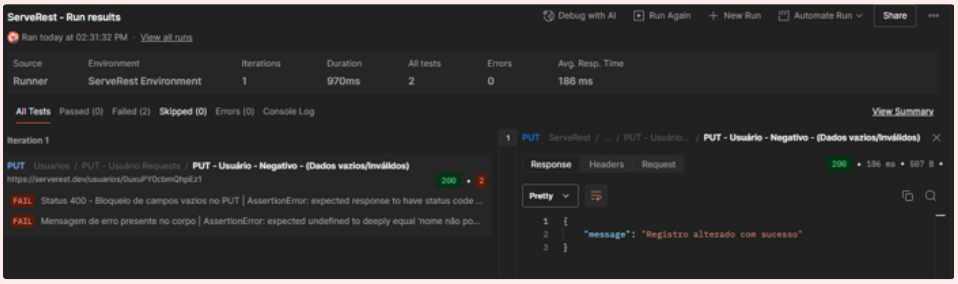


BUG-12 Atualização de usuário permite `nome` e `password` vazios via método PUT

- **Severidade:** Crítica
- **Descrição**:
  Verificar se a API bloqueia a atualização de um registro quando os campos essenciais são enviados como strings vazias, preservando os dados anteriores.
- **Passos para reproduzir:**

1. Identificar um ID de usuário válido (`{{userId}}`).
2. Enviar requisição PUT para `/usuarios/{{userId}}` com campos `nome` e `password` vazios.

- **Resultado Esperado:**
  Status Code: 400 Bad Request com validação dos campos.
- **Resultado Obtido (BUG):**
  Status Code: 200 OK e os dados do usuário são apagados no banco de dados.



BUG-04: Atualização de Produto aceita String em campos Numéricos (PUT /produtos)

Severidade: Crítica

Passos para reproduzir:

1. Identificar um ID de produto válido no sistema (`{{productId}}`).
2. Preparar uma requisição **PUT** para `/produtos/{{productId}}`.
3. No corpo (body) da requisição, enviar os campos com valores entre aspas (Strings):

```
{
  "nome": "Monitor gamer",
  "preco": "500",
  "descricao": "Monitor",
  "quantidade": "10"
}
```

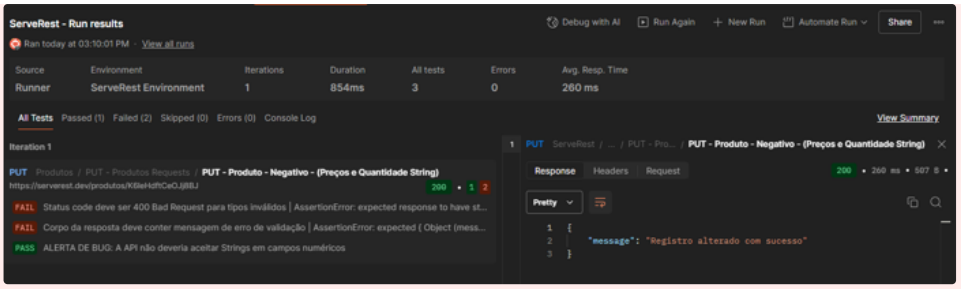
4. Enviar a requisição e observar o retorno.

Resultado Esperado:

- **Status Code:** 400 Bad Request.
- **Mensagem de Erro:** "Os campos 'preco' e 'quantidade' devem ser do tipo numérico/integer."

Resultado Obtido (BUG):

- **Status Code:** 200 OK.
- O sistema aceita os valores como texto, o que pode causar erros em cálculos matemáticos futuros ou falhas de renderização no front-end.



BUG-25: Criação de carrinho aceita quantidade tipo String (POST /carrinhos)

Severidade: Alta

Descrição:

O endpoint de criação de carrinhos (`POST /carrinhos`) falha ao validar o campo `quantidade` . O sistema aceita valores de dados incorretos (String), permitindo a finalização da requisição com sucesso.

Passos para reproduzir:

1. Autenticar um usuário e obter o token.
2. Enviar uma requisição **POST** para `/carrinhos` .
3. No corpo da requisição, inserir um produto com quantidade negativa em formato de texto:

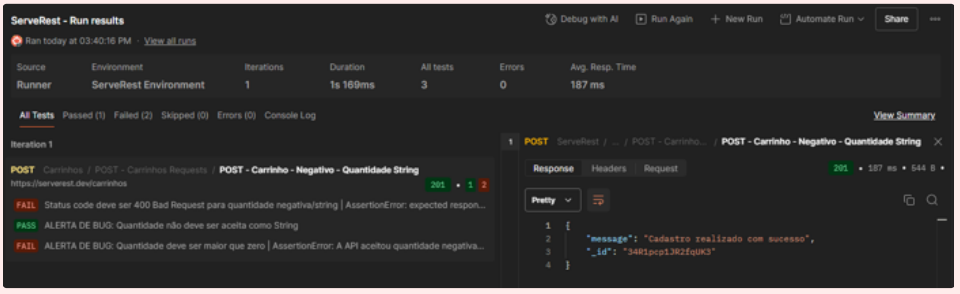
```
{
  "produtos": [
    {
      "idProduto": "{{productId}}",
      "quantidade": "2"
    }
  ]
}
```

Resultado Esperado:

- **Status Code:** 400 Bad Request.
- "Quantidade deve ser um número inteiro positivo."

Resultado Obtido (BUG):

- **Status Code:** 201 Created.
- O carrinho é gerado com saldo negativo, afetando o cálculo total e o inventário.



---

4. Checks Automatizados

- **Ferramenta:** Scripts em JavaScript (Postman Sandbox).
- **Escopo da Automação:**
  - **Positivo:** Validação de login com salvamento dinâmico de Bearer Token.
  - **Negativo:** Validação de mensagem de erro específica para e-mail já cadastrado.
  - **Contrato:** Verificação de Schema básico (presença de chaves `_id` e `message`) e tempo de resposta < 500ms.

---

5. Recomendações e Considerações Finais

- **Segurança:** Implementar validação rigorosa de campos obrigatórios no Backend para evitar usuários fantasmas.
- **Consistência:** Padronizar a tipagem do banco de dados para rejeitar strings em campos financeiros.
- **Próximos Passos:** Realizar testes de carga (Stress) nos carrinhos, pois falhas de estoque podem indicar condições de corrida (Race Conditions).